

Fall 2023

Automatic Presentation Slide Generation Using LLMs

Tanya Gupta
San Jose State University

Follow this and additional works at: https://scholarworks.sjsu.edu/etd_theses



Part of the [Computer Engineering Commons](#)

Recommended Citation

Gupta, Tanya, "Automatic Presentation Slide Generation Using LLMs" (2023). *Master's Theses*. 5444.
DOI: <https://doi.org/10.31979/etd.2x3f-w7uv>
https://scholarworks.sjsu.edu/etd_theses/5444

This Thesis is brought to you for free and open access by the Master's Theses and Graduate Research at SJSU ScholarWorks. It has been accepted for inclusion in Master's Theses by an authorized administrator of SJSU ScholarWorks. For more information, please contact scholarworks@sjsu.edu.

AUTOMATIC SLIDE GENERATION FROM SCIENTIFIC
DOCUMENTS USING LLMS

A Thesis

Presented to

The Faculty of the Department of Computer Engineering

San José State University

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

by

Tanya Gupta

December 2023

© 2023

Tanya Gupta

ALL RIGHTS RESERVED

The Designated Thesis Committee Approves the Thesis Titled

AUTOMATIC SLIDE GENERATION FROM SCIENTIFIC
DOCUMENTS USING LLMS

by

Tanya Gupta

APPROVED FOR THE DEPARTMENT OF COMPUTER ENGINEERING

SAN JOSÉ STATE UNIVERSITY

December 2023

Jorjeta G. Jetcheva, Ph.D.

Department of Computer Engineering

Magdalini Eirinaki, Ph.D.

Department of Computer Engineering

Mahima Agumbe Suresh, Ph.D.

Department of Computer Engineering

ABSTRACT

AUTOMATIC SLIDE GENERATION FROM SCIENTIFIC DOCUMENTS USING LLMS

by Tanya Gupta

Presentation slides are widely used for conveying information in academic and professional contexts. However, manual slide creation can be time-consuming. Our research focuses on automated slide generation, specifically for scientific research papers.

Automating the creation of presentation slides for scientific documents is a rather novel task and hence, there's limited training data available and there also exists the token constraints of language models like BERT, with a maximum sequence length of 512 tokens. In this study, we fine-tune large language models, including Longformer-Encoder-Decoder (supporting sequences up to 16,834 tokens) and BIGBIRD-Pegasus (supporting sequences up to 4,096 tokens). We tackle this task using two approaches, one based on abstractive summarization and other on the hybrid summarization approaches. We use one of the largest dataset available for automatic slide generation of scientific document scientific papers i.e., PS5K.

Our research shows that a model supporting a longer maximum sequence length when working with entire documents performs better. This approach yielded superior results, particularly when the model was trained on section-slide pairs, showcasing higher R2 and RL scores, indicating enhanced coherence compared to other experiments.

ACKNOWLEDGMENTS

I extend my sincere gratitude toward Dr. Jorjeta G. Jetcheva, for her exceptional guidance and unwavering support throughout this research.

I express my gratitude to Dr. Magdalini Erinaki and Dr. Mahima Agumbe Suresh for their valuable contributions in joining the thesis committee, which significantly enhanced the quality of this research.

I also want to thank my family, friends, and peers for their encouragement and camaraderie, which made this journey rewarding.

Lastly, I appreciate San Jose State University for providing access to essential resources.

TABLE OF CONTENTS

List of Tables.....	viii
List of Figures.....	ix
1 Introduction.....	1
1.1 Abstractive approach of slide generation.....	2
1.2 Hybrid approach of slide generation.....	2
2 Background and Related Work.....	5
2.1 Text Summarization.....	5
2.2 Automatic Text Summarization.....	5
2.3 Automatic Text Summarization.....	6
2.4 Abstractive summarization.....	7
2.5 Automatic Slide generation.....	7
2.6 Scientific Article Summarization.....	11
2.7 Multimodal Summarization.....	13
3 Methodology.....	14
3.1 Datasets.....	14
3.2 Dataset Properties.....	15
3.3 Text Preprocessing.....	19
3.3.1 Removing URLs.....	20
3.3.2 Lowercase the text.....	20
3.3.3 Removing the special characters.....	20
3.3.4 Removing the equations.....	20
3.3.5 Removing the whitespace.....	20
3.3.6 Removing HTML Figure and Reference Tags.....	21
3.3.7 Removal of gibberish text.....	21
3.3.8 Slide Stemming.....	21
3.3.8.1 Removal of near duplicate Slides.....	21
3.3.8.2 Removing Slides with just two lines.....	21
3.3.9 Slide-Section Matching.....	22
3.4 Transformer Overview.....	24
3.4.1 Encoder.....	27
3.4.2 Decoder.....	27
3.4.3 Attention Mechanism.....	28
3.4.3.1 Self-attention.....	28
3.4.3.1 Multi-head Attention.....	29
3.4.4 Output Generation.....	29
3.5 Longformer Encode-Decoder.....	32

3.6	BIGBIRD-Pegasus.....	37
3.7	Evaluation Metrics.....	41
3.8	Experimental Platform.....	41
4	Experimental Results.....	42
4.1	Text pre-processing.....	42
4.2	Slide Stemming.....	43
4.3	Matching Algorithm Experiments on pre-trained LED model.....	43
4.3.1	Matching with highest cosine similarity score.....	44
4.3.2	Matching based on cosine threshold.....	44
4.3.3	Matching in a hierarchical manner.....	45
4.4	Fine-tuning the Pre-trained models.....	46
4.4.1	Abstractive approach of slide generation.....	47
4.4.2	Hybrid approach of slide generation.....	50
4.4.3	Comparison with GPT model i.e. gpt-3.5-turbo.....	51
4.5	Comparison to Previous work.....	52
5	Discussions.....	54
6	Limitations and Future Work.....	56
7	Contributions.....	59
8	Conclusions.....	62
	Literature Cited.....	65

LIST OF TABLES

Table 1.	LEDForConditionalGeneration Architecture.....	35
Table 2.	Our Fine-tuned LED model Architecture.....	36
Table 3.	BigBirdPegasusForConditionalGeneration Architecture.....	38
Table 4.	Our Fine-tuned BIGBIRD-Pegasus model Architecture.....	40
Table 5.	Results after Text normalization techniques.....	42
Table 6.	Results after Slide Stemming.....	43
Table 7.	Results from Section-Slide matching on Maximum Cosine Score.....	44
Table 8.	Results from Section-Slide matching on Cosine threshold value.....	45
Table 9.	Results from Section-Slide matching hierarchically.....	45
Table 10.	Results after model evaluation on complete scientific document.....	49
Table 11.	Results after model evaluation on slide-section matching.....	51
Table 12.	Results after model evaluation on complete scientific document (repeated).....	51
Table 13.	Results after model evaluation on complete scientific document on gpt-3.5-turbo.....	52
Table 14.	Results from Fine-tuned models.....	55

LIST OF FIGURES

Fig. 1. Processing Complete Long Scientific Documents.....	2
Fig. 2. Processing Segments of Scientific Documents into Sections.....	4
Fig. 3. Number of articles published each year in PS5K.....	16
Fig. 4. Distribution of Paper token length in PS5K.....	17
Fig. 5. Distribution of Slide token length in PS5K.....	17
Fig. 6. Presentation Slide in XML format in PS5K.....	18
Fig. 7. Paper in XML format in PS5K.....	18
Fig. 8. Aligning the scientific document section to a slide.....	23
Fig. 9. Transformer architecture.....	25
Fig. 10. Transformer's Encoder.....	27
Fig. 11. Transformer's Decoder.....	28
Fig. 12. Building blocks of LED attention mechanism.....	33
Fig. 13. LED tokenizer applied to one sample paper.....	36
Fig. 14. Building blocks of BigBird attention mechanism.....	37
Fig. 15. Big Bird Pegasus tokenizer applied to one sample paper.....	40
Fig. 16. Distribution of pre-processed scientific document token length in PS5K.....	48
Fig. 17. Distribution of pre-processed slide token length in PS5K.....	48
Fig. 18. Results from previous work.....	53

1. INTRODUCTION

Presentation slides are a frequently used format for presenting information in academic and professional settings. In addition, when a slide version of information (such as a research paper or article) is available, it is often the version of choice for knowledge workers to familiarize themselves with the information. However, the manual creation of presentation slides can be labor-intensive. Instead we explore the use of automated slide generation to create initial drafts of presentations based on documents, focusing on (science) research papers.

Current summarization models are trained on books or news articles and are not well-suited for the complex structures and specialized language found in scientific documents. Although some models are designed for scientific papers, they primarily aim to generate abstracts, which is a less challenging task compared to creating presentation slides.

Automating the creation of presentation slides for scientific documents is an underexplored area, mainly due to the limited availability of suitable training data for summarization models and the constraints posed by most of the language models, like BERT, which have a maximum sequence length support of 512 tokens, making it challenging to handle lengthy content.

To achieve this task, we fine-tune pre-trained, large summarization models like Longformer-encoder-decoder, which can handle sequences up to 16,834 tokens, and BIGBIRD-Pegasus, with support for sequences up to 4,096 tokens. We utilize abstractive

summarization and large language model generation techniques to automatically generate slides from scientific papers in PS5K dataset.

We use two different approaches to finetune our models:

1.1 Abstractive approach of slide generation

As shown in Fig 1., after text-preprocessing, we fine-tune the language models by inputting the complete scientific document and its slides. We aim to make the model learn the style and language of the slides. After the model is fine-tuned, and generates the slides from unseen data, we create a presentation slide from the generated slide text using a python package called python-pptx.

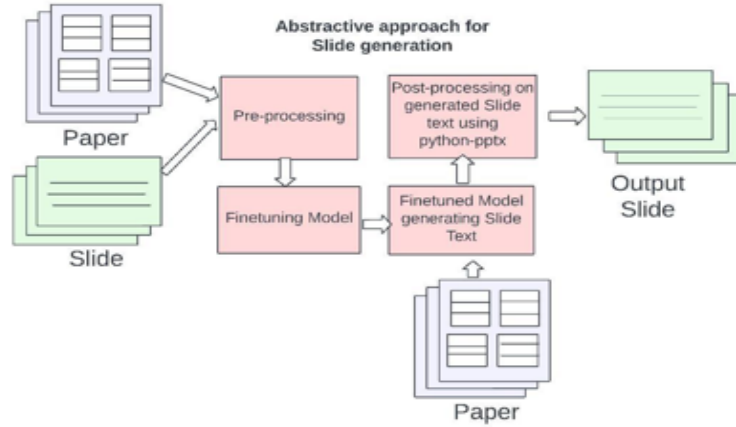


Fig. 1. Abstractive approach of slide generation

1.2 Hybrid approach of slide generation

In this approach, in alignment with [1], we operate under the assumption that a slide can serve as an abstractive summary for a section, and correspondingly, each section can have an associated slide. We classify this method as hybrid summarization since we exclusively choose sections and slides that can be aligned with each other for fine-tuning, excluding other sections that are not included in this process.

As shown in Figure 2, we initiate the process with text preprocessing (refer to Section 3.3). Subsequently, we align the sections of the scientific paper with slides based on textual embeddings, which involves computing cosine similarity scores. We have adopted this approach because, in the first method, despite the utilization of large language models, the text is still truncated due to the model's inherent limitation in processing exceptionally long documents (for instance, the Longformer Encoder-Decoder's token limit is 16834 tokens, and the Big Bird Pegasus's token limit is 4096 tokens). Furthermore, resource constraints only allowed us to fine-tune the Longformer Encoder-Decoder on 9500 tokens for input and 950 tokens for output. While we could fully leverage the input token limit of the Big Bird Pegasus model, the self-attention nature of the decoder results in a quadratic growth in time and memory usage, permitting us to use only 1024 tokens for the output token limit.

As a solution to these challenges, we made the decision to segment the scientific documents into sections and align them with slides. We process the input sections and slides sequentially to preserve their order. Additionally, we explore three supplementary approaches to achieve alignment between sections and slides (refer to Section 3.3.9).

We fine tune the model on section and its aligned slide, this approach saves a lot of time, but relies heavily on the accuracy of textual similarity score. After, we fine-tune the model, we generate the slide for each section and generate a presentation file using a python package called python-pptx.

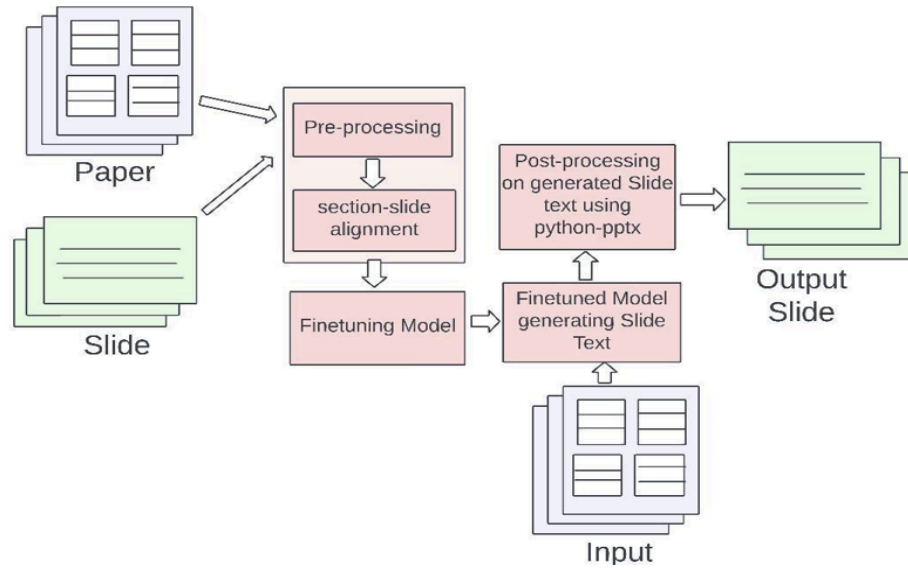


Fig. 2. Hybrid approach of slide generation

Our research demonstrates that when working with entire documents the model that supports longer maximum sequence length i.e. LED, there is a higher unigram overlap i.e. R1 scores. However, when the model is trained using section-slide pairs, it exhibits improved R2 and RL scores. This indicates that the generated slide text maintains a better sequence of words and is more coherent with the reference slide text. This approach leverages a larger volume of training data, achieved by segmenting the input data into sections and avoiding text truncation. Sections can easily fit into available computing resources and model token length. These findings make a valuable contribution to automated presentation generation in academic and professional contexts.

2. BACKGROUND & RELATED WORK

Since slide generation is a form of summarization, we start by reviewing the definition and state of the art in summarization. We then discuss current approaches for automated slide generation.

2.1 Text Summarization

In [2], summary is defined “as a text that is produced from one or more texts, that conveys important information in the original text(s), and that is no longer than half of the original text(s) and usually significantly less than that”. They [2] further added that “main goal of a summary is to present the main ideas in a document in less space”, which fits to our task as well, as we aim to generate the presentation slides from a scientific document by either inputting the complete scientific document or dividing scientific document based on its sections to produce the slide text. They also said that the summaries can be of two types, i.e. indicative summaries(what is text about, without specific content) and informative summaries(the shorter version of the text). We focus on informative summaries where we treat slide deck as the summary of the scientific document.

2.2 Automatic Text Summarization

Automatic Text Summarization (ATS) has gained significant attention over the last two years due to the exponential growth of textual data on the Internet and extensive archives of documents like news articles, scientific papers, and legal texts. Manual summarization is resource-intensive and often infeasible given the vast volume of text [3].

Two primary NLP approaches are used for automatic text summarization: extractive and abstractive summarization. Extractive methods select key sentences from the input document(s) and arrange them to form the summary. In contrast, abstractive methods represent the input document(s) in a middle-level format and then create the summary using sentences that may differ from the original ones.

2.3 Extractive Summarization

In neural models for extractive summarization, the task is formulated as a sentence classification problem where the neural encoder is used to create sentence representations, and a classifier decides which sentences to include in the summaries. For instance, an early example is SummaRuNNer [4], which employs a neural encoder based on Recurrent Neural Networks (RNNs). Another approach, REFRESH[5], introduces reinforcement learning to rank sentences for extractive summarization, improving summary quality based on the ROUGE metric.

In the study presented in [6], extractive summarization is performed by fine-tuning the BERT model, where the model is trained to classify sentences as either included or excluded in the summary based on their importance in capturing document information. This fine-tuned model is then used to select the most salient sentences for the final extractive summary.

Many researchers have leveraged machine learning techniques to determine the importance of sentences. These generally have rank sentence importance: regression in studies presented in [1], [7], [8] and deep neural networks in [9]. They also use another strategy to perform sentence selection using integer linear programming [1], [7], [8], [9].

However, these methods rely on extractive summarization, where they extract raw sentences and phrases from documents as generated slide content.

2.4 Abstractive summarization

In earlier work [1], [10] were early adopters of the neural encoder-decoder architecture for text summarization. A study presented in [11] conducted abstractive summarization using a neural attention-based model that generates summaries by attending to relevant parts of the source document. Another study in [12] improved upon this model by introducing a pointer-generator network (PTGEN) that enables copying words from the source text and a coverage mechanism (COV) to track previously summarized words. Another study [13] proposed an abstractive system where multiple agents (encoders) collectively represent the document with a hierarchical attention mechanism for decoding, and their Deep Communicating Agents (DCA) model is trained end-to-end using reinforcement learning. PEGASUS, a pre-training method with extracted gap sentences, for abstractive summarization, allowing the model to generate concise and coherent summaries by filling in information gaps in the source text [14].

Text summarization using extractive summarization has become a broad research topic and reaching toward maturity, in comparison to abstractive summarization, which is under investigation. Especially the summarization of scientific material.

2.5 Automatic Slide Generation

Automatically generating presentation slides for academic papers is gaining increasing interest. These generated slides serve as draft versions, assisting presenters in expeditiously preparing their final, formal presentation slides. In a study presented in

[15], they took on the challenge of automating the creation of presentations from documents enriched with semantic annotations. They employed standard text documents, which had undergone a semi-automatic annotation process using GDA tagset to deduce semantic relationships between sentences. These relationships included the identification of noun phrases, verb phrases, grammatical connections such as subjects and verbs, thematic roles like agents, patients, recipients, and rhetorical relationships like causes and elaborations.

In another study [16] developed a method for creating slides from raw text. They approached this by treating clauses and sentences as discourse units and identifying various relations such as contrast, list, additive, topic-chaining, elaboration, cause, and example between them. Specific clauses were designated as topic parts, while others were categorized as non-topic parts. These extracted topic and non-topic components were then strategically placed on the slides according to the detected discourse structure. The authors also introduced heuristic rules for generating slides from these topic and non-topic segments.

The studies [15], [16] are dependent on language specific parsers to identify pre-defined relationships and text selected for presentation is randomly selected, which could potentially lead to incoherent text.

In a method called SlideGen, developed in [17], they addressed the task of automatically generating slides for scientific documents by converting it to a summarization problem using a dual approach. They employed an extractive technique, where they identified essential sentences from research papers, and an abstractive

approach, where they fine-tuned pre-trained language models on scientific document's sections and their matching slides, to understand the language commonly used in creating presentation slides. Additionally, they introduced a dataset called PS5K, comprising 5000 paper-slide pairs within the domain of computer and information science.

The study presented in [18], [19], [20], concentrates on aligning slides with paper sections. Hayama et al. employ a Hidden Markov Model (HMM) for slide-to-section alignment, while study in [20] explores various models based on TF-IDF term scores. The study in [19] employs machine learning techniques to establish a monotonic alignment between paper sections and slides

PPSGen[1] creates presentation slides for scholarly documents by leveraging the PPSGen method, which involves automatic summarization and content selection to condense academic papers into coherent slide presentations. They used a smaller dataset of 1200 paper-slides pairs. They applied Support vector regressor for sentence ranking and ILP to select important sentences, another generates slides by phrases extracted from papers. The model learns the importance of each phrase and the hierarchical relationship between a pair of phrases to make bullet points to determine their places in the slide. However, the model was tested on a limited set of only 200 papers.

The study presented in [7] tackles the challenge of automating slide creation from scholarly documents. It employs a phrase-based methodology to extract crucial content from academic papers and compose presentation slides. Their model is trained on a small dataset of 175 paper-slide pairs.

The study presented in [21] introduces a topic-aware framework for slide generation, incorporating four key topics ("Contribution," "Dataset," "Baseline," "Future Work") by extracting relevant sentences. Their approach utilizes a mutual learning algorithm where a neural sentence selector is trained to choose sentences, and a log-linear classifier incorporates prior knowledge as features.

In our research, we embark on the challenge of automating the creation of presentation slides by transforming the task into abstractive summarization. We achieve this by fine-tuning pre-trained language models, namely, the Longformer-Encoder-Decoder and BIGBIRD-Pegasus. These models have been pre-trained on a vast scientific dataset, which is designed for generating abstracts from scientific documents. This dataset is sourced from arXiv, a prestigious preprint repository and open-access platform housing scholarly research articles. The models and scientific dataset is available on the Hugging Face platform.

Our objective is to further fine-tune these models using the PS5K dataset. We adopt a dual-pronged approach:

Firstly, we employ a large transformer model to process complete scientific documents, thereby generating text that resembles slides.

Secondly, due to computational constraints and even large language model's incapability to process large documents, we segment the scientific documents by section and align them with slides based on textual similarity. This methodology closely aligns with the approach presented in [17], where scientific document sections are aligned with

slides by maximizing textual similarity and fine-tuning the model. In our work, we undertake this alignment in three different ways:

1. We compute the cosine similarity score for each section-slide pair, relying on text embeddings generated using the DistilBERT-based model from Sentence Transformers' pre-trained model.
2. Next, we introduce a threshold value for the cosine similarity score to identify suitable section-slide pairs, experimenting with values ranging from 0.4 to 0.6 in increments of 0.05.
3. We try to align sections with slides in a hierarchical fashion, seeking insights the way the slide deck is organized in this dataset. For each section, we identify the top three slide matches. Instead of rigidly adhering to the best match, we explore the possibility of matching a section with a slide whose index surpasses the slide matched to the preceding section. We ensure that the similarity score between the two should not deviate by more than 0.07 from the maximum similarity score to avoid bad matches.

This multi-faceted approach enables a comprehensive exploration of section-slide alignment, enhancing the quality and relevance of automatically generated slides.

2.6 Scientific Article Summarization

The initial exploration of scientific article summarization can be traced back to a study in [22], who pioneered the use of a supervised Naive Bayes classifier to identify crucial content for summaries. Subsequently, in [23] advocated for the advantages of incorporating citations in the analysis of scientific work. In 2015, another study presented in [24] introduced a search-oriented approach to pinpoint relevant segments within

reference papers for citation inclusion. Building on this, another work in both 2008 [24] and 2013 [25], harnessed citations to construct article summaries. Their methodology involved hierarchical agglomerative clustering of citations to maximize coherence and the selection of pivotal sentences from each cluster for the ultimate summary.

In [26] distinctive approach combines the utilization of citation-context and the analysis of an article's discourse structure to enhance the quality of generated summaries. By considering the relationships between citations and the main article and by delving into the structural elements of the article, the paper yields summaries that are more contextually relevant and informative. The methodology involves the extraction of citation-context information and the application of discourse structure analysis, resulting in summaries that effectively capture the core content of scientific articles. This contribution represents a significant advancement in the field of scientific summarization, offering a holistic and context-aware approach.

In 2018 [27] introduced a system designed to autonomously create article abstracts in Indonesian. Their method encompasses four key steps. Initially, preprocessing activities, including sentence extraction, case folding, tokenization, filtering, and stemming, ready the input text for the subsequent stages. Next, they apply Term Frequency-Inverse Document Frequency (TF-IDF) calculations to each term in the preprocessed text. Using cosine similarity and vector space modeling (VSM), they assess text similarity with 20 TF-IDF keywords, ranking sentences based on these scores. Ultimately, the final abstract comprises the top ten sentences. To evaluate the system, manual abstracts were compared to its output, revealing significant overlap, as the system-generated abstracts share three

or more sentences with the manual abstracts. This overlap stems from the inclusion of unique terms in the author's abstract that are absent in the article body.

2.7 Multi-modal Summarization

There has been much work done in recent years on multi-modal summarization. The study presented in [28] presented a multi-modal summarization task, which takes new images and generates pictorial summaries. They achieve this by proposing a multimodal attention model to jointly generate text and most relevant images. The importance of images is determined by the visual coverage vector. They also collect a large scale dataset from Daily Mail website.

In [29], proposed an extractive summarization, multi-modal summarization method that automatically generates textual summary, given a set of documents, images, audios and videos related to a specific topic. They also released a multi-modal summarization corpus in English and Chinese., such that they selected 50 news topics in both the languages.

DOC2PPT [30] does multimodal summarization with extractive summarization and slide generation from scientific documents. It leverages both textual and visual elements within these documents to automatically create presentation slides. They propose a hierarchical sequence-to-sequence approach by using the inherent structures within documents and slides and incorporate paraphrasing and layout prediction modules to generate slides.

Our work differs from their approach in that we focus on the text-only portion of the presentation.

3. METHODOLOGY

3.1 Datasets

In [17], introduced a dataset called PS5K, a corpus of 5000 paper-slide pairs compiled from conference websites like www.usenix.org and <https://www.aclweb.org>. We use this dataset, as text is already extracted into XML files and hence make it easy to use and makes the process of training the model faster. In [9], they use PS5K dataset to generate slides extractively at document level using BertSum and abtractively at section level using BART and compare the results using Rouge score.

These are in the domain of computational linguistics, system and system security. The majority, i.e. about 75% of papers are published between 2013 and 2019. The crawled papers are in PDF format and slides are either in PDF or PPT(or PPTX) format. In another study [31], they use GROBID to extract metadata and text from scientific papers, and Apache Tika is used to convert presentations into XML format.

This study does document-level slide generation and section-level slide generation using large Longformer Encoder-Decoder and BigBirdPegasus models. These models are pre-trained on arxiv papers in scientific_papers dataset available on Huggingface.

Another dataset introduced [30], introduced a large dataset of 5,873 pairs of scientific documents and slide deck. These were collected from academic proceedings, focused mainly on three research communities: computer vision, natural language processing and machine learning. The crawled papers are in the PDF format and the slides are in image file format. We focus on generating text-based presentation slides. The text extraction

will need sophisticated tools and significant effort, therefore, we use PS5K which provides the scientific document and its slide text extracted into the XML file.

We reviewed other datasets but found that they have shortcomings relative to the goal of our work. Scientific Document Summarization and Automatic Slide generation using scientific documents has recently started gaining interest. Therefore, there are not many datasets available, even if they are not enough to train the model.

The CL-SciSumm 2018 [32] summarization task aimed at bringing together the summarization of the community to address challenges in scientific article summarization. It explores scientific document summarization in the domain of computational linguistics. It was conducted at TAC 2014 as part of the larger BioMedSumm Task. This task had only 40 NLP papers with human-annotated reference summaries. Additionally, ScisummNet [33] expanded the CL-Scisumm [32] to 1000 scientific articles. TalkSumm [34] proposed a dataset of 1716 papers and their videos and created a dataset of paper summaries. These summaries were generated by utilizing the videos of talks at scientific conferences. Both of these datasets are rather small with only about 1K documents each.

3.2 Dataset Properties

PS5K is a 5000 paper-slide pair from conference websites, e.g., usenix.org and aclweb.org. These are in the domain of computational linguistics, system and system security.

Fig. 3. shows the number of scientific articles published each year. The majority, i.e. about 75% of papers are published between 2013 and 2019. All the scientific documents

and its corresponding slides are available in PDF format and PPT format respectively, and have been extracted using GROBID and Apache Tika respectively.

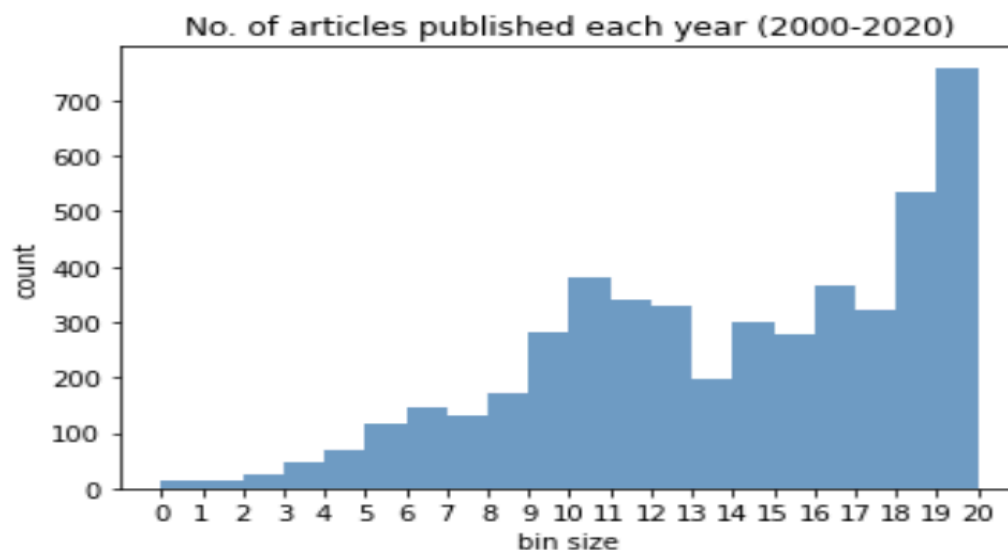


Fig. 3. Number of articles published each year in PS5K

Fig. 4. shows the number of tokens in the training articles. The maximum document size is approximately 20,000 tokens. Again, due to limited computing resources, the maximum encoder length has been set to 9500 tokens for Longformer-Encoder-Decoder(LED) and due to BIGBIRD-Pegasus models' token limit set to 4096 tokens.

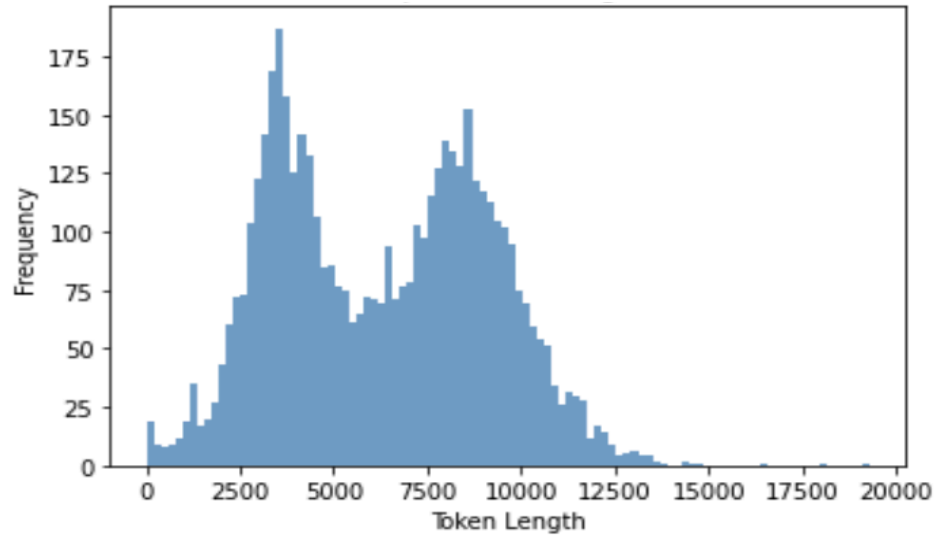


Fig. 4. Distribution of scientific document token length in PS5K

Fig. 5. shows the number of tokens in slides in our training corpus. We can infer that the maximum number of tokens per slide to be 5000. Unfortunately, due to limited computed resources, we set the decoder token limit to 950 tokens for Longformer-Encoder-Decoder(LED) and 1024 tokens for the BIGBIRD-Pegasus model.

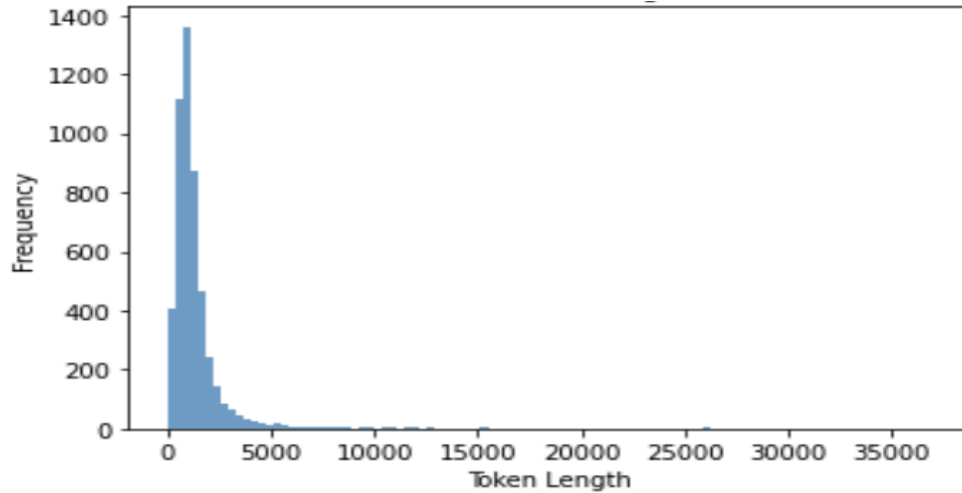


Fig. 5. Distribution of Slide token length in PS5K

In a study [9], extracted textual content from scientific documents and its slides into XML files. The principal benefit of this approach is it allows parsing the scientific

documents to different topical sections. GROBID achieved best results for metadata and references extraction, though it struggled to extract lists, footers and equations [35].

Within the context of these XML files, the structuring of content is as follows:

As shown in Fig. 6., Each section in the scientific document is encapsulated within div tags. These div tags further contain a head tag, serving the purpose of denoting the section's title, while the section's text is systematically organized within p tags.



Fig. 6. Presentation Slide in XML format in PS5K

As shown in Fig. 7., In the case of presentation slides, their content is similarly organized using div tags. Each line or unit of text on a slide is encompassed within p tags, ensuring a coherent and structured representation of the slide's content.



Fig. 7. Scientific document in XML format in PS5K

3.3 Text Preprocessing

The initial and critical phase of Natural Language Processing (NLP) model training is dedicated to data preprocessing. This phase's primary objective is to remove noise from the text, thereby enhancing the learning process for the models. The quality of the input data significantly influences the models' performance and the reliability of text generation experiments and their results. Our empirical findings support the hypothesis that models perform better when provided with preprocessed text rather than raw, unprocessed text. It's important to note that the sequence of preprocessing steps plays a key role in shaping the data's format during experimentation.

In our approach, we chose not to utilize certain text normalization techniques, such as Stemming or Lemmatization. While these methods are commonly used to reduce words to their base or root forms, in scientific and technical documents, specialized vocabulary and domain-specific terminology are frequently encountered, which may not be present in standard dictionaries. These terms are essential for accurately representing the content and nuances of scientific texts. This decision underscores the significance of customizing

preprocessing steps to the unique characteristics and objectives of each NLP project, ensuring that the resulting data is well-suited for the intended analyses and model training.

Regarding our dataset, the pairs of scientific papers and slides have already been extracted from PDF and PPT formats and converted into XML files. We proceed to parse these XML files for both the scientific paper and its slides, applying the following preprocessing techniques:

3.3.1 Removing URLs

URLs can be quite long and contain special characters. The removal of URLs help models focus on significant data, reduce noise.

3.3.2 Lowercase the text

It helps in standardizing the text's case such that the model can learn to recognize the words regardless of the letter case, which also helps in better generalization.

3.3.3 Removing special characters

The paper and slide text has been parsed from XML files, and sometimes, parsing errors can lead to generation of long strings of random special characters, which degrades the quality of the training data.

3.3.4 Removing equations

Scientific text contains a lot of equations with different symbols, punctuations. Oftentimes, these symbols are not parsed properly leading to inconsistency between actual and parsed data, so equations have been eliminated from the text to maintain the quality of the data.

3.3.5 Removing whitespace

Removing excess whitespace ensures consistent text formatting, which is crucial for tokenization and tokenization greatly impacts the model performance.

3.3.6 Removing HTML Figure and References tag

All the figures and references tags are eliminated as we focus on creating text-based slides. The user can manually add more information in the form of figures or references later, if needed.

There has been additional preprocessing done on the slides data

3.3.7 Removal of gibberish text

Extracting data via OCR tools can sometimes result in parsing errors, leading to unintelligible content. As a solution, we remove lines in a slide if they consist solely of symbols or special characters.

3.3.8 Slide Stemming

3.3.8.1 Removal of near duplicate slides: Many slides contain animation such that there's duplication of text between preceding and current slide. These near-duplicate slides are redundant and only the final slide is kept. These have been removed based on textual embeddings by computing cosine similarity scores. In a study[32] the preceding slides are discarded, if the supplication level is 80% or above on their dataset. To decide a similarity threshold on our chosen dataset, we performed the experiment on three different threshold values like 0.7, 0.8 and 0.9.

3.3.8.2 Removing slides with just one line: We filter out slides with only one line. Our methodology adopts a hierarchical two-level structure, with each slide comprising a title and description segmented into lines. Consequently, a minimum of two lines is expected in each slide.

3.3.9 Slide-Section Matching

To address constraints stemming from limited computing resources and the model's token limit, we divide the paper and slides into smaller chunks based on their respective sections and slides. Subsequently, we establish alignments between the paper's sections and corresponding slides by matching them using textual embeddings. Each matched slide serves as a concise summary of its corresponding paper section. This matching process relies on textual similarity, computed through cosine similarity scores.

For generating the textual embeddings for both paper and slide, we utilize Sentence-Transformers, which is built upon Sentence-BERT (SBERT). SBERT[36] is a modification of the pretrained BERT network, employing Siamese and triplet network

structures to derive semantically meaningful sentence embeddings that can be compared using cosine similarity. Sentence-Transformers is based on Pytorch and Transformers and offers a collection of pre-trained models. These models have been extensively trained and evaluated on diverse datasets for sentence encoding and semantic search tasks. In our work, we specifically employ the all-distilroberta-v1 model, which is based on distilroberta-base and supports a maximum sequence length of 512 tokens. This model has undergone training on a vast dataset comprising over 1 billion training pairs across various tasks, including semantic search.

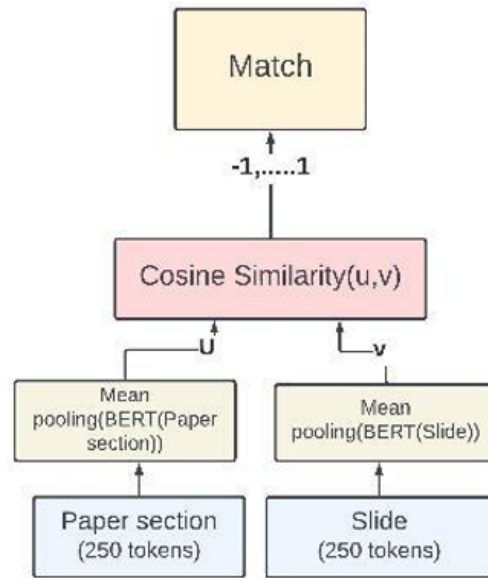


Fig. 8. Aligning the scientific document section to a slide

Fig. 8. illustrates the process of aligning paper sections and slides. Initially, paper sections are segmented to ensure compatibility with the model's maximum sequence length. For simplicity, both paper sections and slides are divided into chunks, each consisting of 250 tokens. These chunks are then input into the BERT model (distilroberta-base) to generate textual embeddings. Subsequently, a pooling operation is

applied to create fixed-size sentence embeddings, denoted as 'u' and 'v.' These embeddings facilitate the computation of cosine similarity, enabling each chunked paper section to be compared against all the slide embeddings and, ultimately, matched with the slide based on their cosine similarity scores

We employ a multi-faceted approach to align sections in scientific papers with their respective slides. Our initial strategy entails identifying the most suitable slide by selecting the top matching candidate, i.e., the one with the highest cosine similarity score for a given paper section.

Subsequently, to refine the quality of these matches, we introduce a threshold value within the aforementioned approach. This threshold signifies the point at which a match is deemed acceptable. We conduct experiments by systematically varying this threshold from 0.3 to 0.6 in 0.05 increments and evaluate the resulting outcomes.

In our third approach, we consider the structural arrangement of the slide deck in relation to the format of the scientific paper, seeking any valuable insights into the organization of these slide decks. For each scientific paper, we identify the top three slide matches using the criteria established in our second approach. These matches exceed a predefined threshold value. We record the index value of the matched slide with respect to the corresponding paper section and attempt to match subsequent paper sections with index values equal to or greater than the index value of the previous paper section's slide match.

Notably, we observe variations in the cosine similarity scores among the top three matches, prompting us to ensure that, in the interest of maintaining the quality of matched

slides, the selected slide's cosine similarity score does not deviate significantly from the highest score. To address this, we set a threshold of 0.07 and carry out experiments within this paradigm as well.

3.4 Transformer Overview

In this section, we overview transformers which are a core machine learning element of our models. Transformers-based language models[37], [38], [39] were proposed in 2017, and have been widely popular and successful in all fields of artificial intelligence, like natural language processing, computer vision and audio processing. Transformers are most sought after architecture in natural language processing, especially pretrained language models(PTMs) [40] and have achieved state of art results in various tasks like question answering, summarization etc.

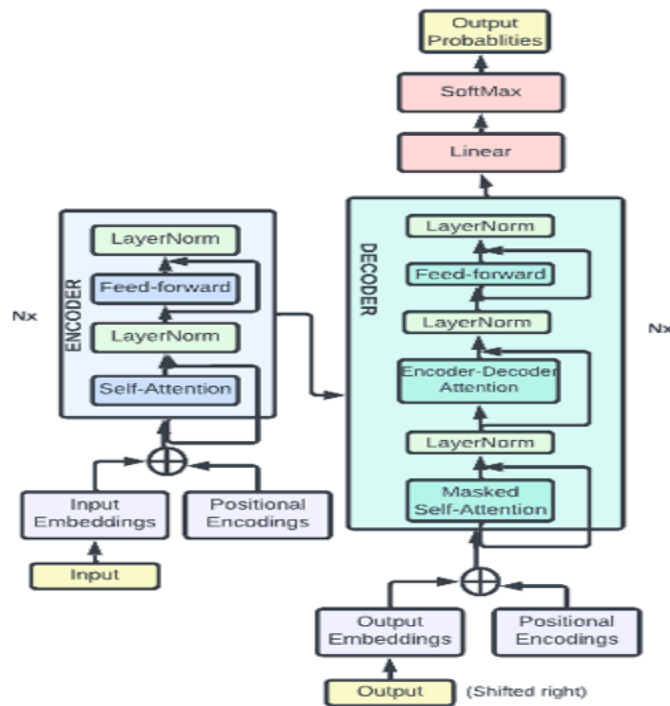


Fig. 9. Transformer architecture

As illustrated in Fig. 9., transformers consist mainly of two components called Encoder and Decoder. Encoder consists of a self-attention module and feed -forward network.

Decoder consists of self-attention module, encoder-decoder attention and feed forward network.

Multiple layers are stacked for both the encoders and decoders, each with its unique embeddings. These encoders and decoders are identical and connected in sequence. Moreover, they incorporate Residual skip connections around both layers and include two LayerNorm layers for improved stability.

The input to both the encoder and decoder consists of two components: embeddings and position encodings. The embedding layer serves to encode the meanings of words by mapping individual input words to embedding vectors. Meanwhile, the position encoding indicates the positions of these words within the sequence.

In contrast to traditional RNNs, where sequences are processed sequentially, transformers process words in parallel. Consequently, transformers introduce position information separately through the use of position encodings.

Transformers employ two embedding layers: one in the encoder for processing the input and another in the decoder for handling the target sequence. Both embedding layers, along with the positional encoding layers, operate on matrices. These matrices are shaped to accommodate word IDs, organized as "Number of Samples x Sequence Length." Each

word ID is transformed into a word vector with a length equal to the embedding size, resulting in dimensions of "Number of Samples x Sequence Length x Embedding Size."

The positional encoding shares a similar matrix shape, allowing it to be added to the embedded matrix while maintaining an encoding size that matches the embedding size. The matrix's shape, as established by the embedding and positional encoding, is preserved until the final layers, where it is reshaped for further processing.

3.4.1 Encoder

In the encoder stack, each encoder receives input from the previous encoder. Inside an encoder, it passes it to the multi-head attention self-attention layer, which is further fed to the feed-forward layer. The output of each encoder is fed into each decoder.

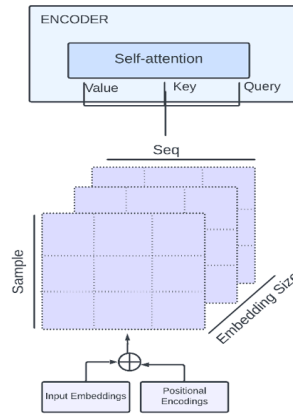


Fig. 10. Transformer's Encoder

3.4.2 Decoder

In the decoder, it takes input from the preceding decoder step. Within the decoder, this input undergoes processing within the Multi-head Self-Attention layer. Notably, unlike the encoder, the decoder selectively attends to both the current and past positions

while deliberately masking any information related to future positions. Additionally, the decoder incorporates an extra Multi-head Attention layer, known as the Encoder-Decoder Attention layer. This layer combines information from the encoder's output and the input from the previous decoder step. Finally, the resulting output is passed to the feed-forward layer for further processing.

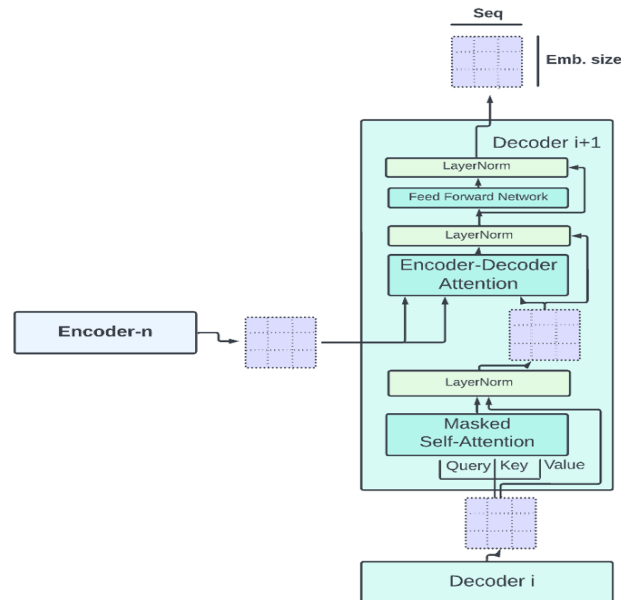


Fig. 11. Transformer's Encoder

3.4.3 Attention mechanism

The popularity of transformers stems from their attention component, which empowers the network to capture contextual information across the entire sequence. The Attention layer receives three inputs: query, key, and value. These inputs undergo distinct linear transformations, encoding representations for every word, and attention scores are computed between each word and every other word in the sequence. In this context, you

can view the query as the word for which the attention score is being calculated, while the key and value represent the words to which attention is directed.

3.4.3.1 Self-attention: The attention score is a dot product of the query, key and values, except that it's also divided by the scaling factor called $\sqrt{d_k}$, followed by application of the softmax function and this called Scaled Dot Product Attention. The principle of dot product i.e. higher the alignment between two vectors, their attention score will be higher, and vice versa.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad [37]$$

It's used in three places:

1. Self-attention in encoder - the input sequence attends itself
2. Encoder-Decoder Attention - target sequence attends to input sequence.
3. Self-attention in decoder - target sequence attends itself.

In case of the encoder and decoder self attention, all three values are either from input or target sequence, and an attention mask is used to ensure that padding doesn't contribute to the attention computation. While, in encoder and decoder attention, the query is the output of decoder self-attention and key and value is the output of the last encoder, and an attention mask is used to prevent decoders from learning about the output in advance.

3.4.3.2 Multi-head Attention: Attention scores are computed h times, which are combined together to produce a final attention score. Each attention function is called head and is denoted by h . This helps the model in learning more contextual information about the words.

$$\begin{aligned}
\text{MutliHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O & [37] \\
\text{Where } \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)
\end{aligned}$$

3.4.4 Output Generation

In the transformer architecture, the output from the last decoder is directed to the output component. It undergoes processing through a linear layer, which projects it into word scores, with a separate score assigned to each word at every position in the sequence. Subsequently, the Softmax layer is applied to these scores, transforming them into probabilities. At each position, the index with the highest probability is selected, mapping it to a corresponding word in the vocabulary at that position within the sequence. These words collectively form the final output sequence.

The training process for a transformer involves several key steps. The input sequence is initially converted into embeddings, combined with their respective positional encodings, and then forwarded to the encoder. Similarly, the decoder receives the target sequence embeddings along with their positional encodings, with the additional step of prefixing the sequence with a start-of-sentence token.

The encoder processes the provided embeddings, generating an encoded representation, and subsequently passes this representation to the decoder. The decoder, armed with its own embeddings and the processed encoded representations from the encoder, produces an encoded representation of the target sequence.

This final encoded representation is transformed into word probabilities, resulting in the generation of the ultimate output sequence. This output is then compared to the target sequence, and a loss is calculated to determine the dissimilarity between the generated

and desired outputs. The loss value is crucial for the training of transformers, as it guides the generation of gradients used during backpropagation, ultimately refining the model's parameters and enabling it to learn and improve from the training data.

During inference, only input sequence is fed to the model, and the encoder generates embeddings along with positional embeddings and fed to the decoder. Since decoder has no target sequence, it generates embeddings from empty strings which are prepended with the start-of-sentence token. The decoder starts processing its own embeddings along with the encoded representations from the encoder, and an output sequence is generated. The last word from the sequence is appended to the decoder's input sequence. So, the new sequence becomes a start-of-sentence token and a new output sequence, this is fed to the decoder and this continues until the end-of-sentence token is predicted by the model.

There are three kinds of the transformer architecture- encoder only, decoder only and encoder-decoder. In this work, an encoder-decoder variant of Transformer is used for seq2seq modeling. The benefit is that the model is equipped with the ability to perform natural language understanding and generation. BART is constructed from bi-directional encoder like BERT and auto-regressive decoder like GPT. While the transformers are powerful, the memory and computation requirements of self-attention grows quadratically, making it expensive to process long documents. Currently, a benchmark dataset with an average source document length that exceeds 3,000 lexical tokens could be well-considered as “long documents” [41] due to the fact that most existing state-of-the-art summarization systems (e.g., pre-trained models) are limited to 512 to 1,024 lexical tokens only.

Despite Transformers popularity, their core limitation is quadratic dependency on sequence length due to its full attention mechanism. Recently, modified transformer architectures like Longformer and BigBird have been introduced which reduce model complexity from quadratic to linear.

As Mentioned in [14], [42], pre-training helps in generative tasks. We warm-start from publicly released checkpoints of the Longformer-Encoder-Decoder and BIGBIRD-Pegasus model trained on arXiv paper in scientific_paper dataset on hugging face saving significant amounts of compute time. We fine-tune these models on our downstream task of generating presentation slides from scientific documents, such that we treat slides as the abstract summary, also considering that the technical papers have hierarchical structure.

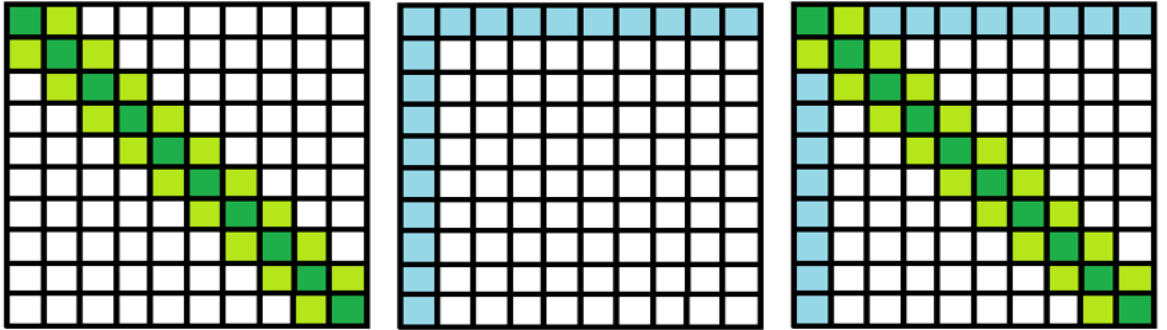
3.5 Longformer-Encoder-Decoder

The LED (Longformer-Encoder-Decoder) model is a transformer-based language model developed by the Allen Institute for AI (AI2) that achieves state-of-the-art performance on a range of natural language processing tasks, including long-range summarization, question answering. The LED model is available on a hugging face in base and large size, with 6 and 12 layers in both encoder and decoder stacks respectively. Though encoder only models like BERT and T5 have achieved state-of-the-art results on summarization tasks, Longformer-Encoder-Decoder is sequence-to-sequence model that has the ability to handle long documents unlike its counterparts and has an efficient attention mechanism. Instead of using full-attention in the encoder, Longformer-Encoder-Decoder uses an efficient local and global attention pattern. The

decoder uses full self-attention to all encoder tokens and previous decoded locations. It uses a sparse attention pattern which is a combination of a windowed local-context self-attention and an end task motivated global attention. Local attention is used for building local contextual embeddings, and it's applied on neighboring tokens while global helps in building full representations.

LED has been initialized from BART, and follows the same architecture in terms of number of layers and hidden units. The difference between BART and LED is BART's 1K tokens are repeatedly copied 16 times to support the new position embedding of 16K [43].

The global attention is added to pre-selected locations only, depending on the task. For example, for summarization, it's added to the start of every sequence. To increase the attention span of each token, LED uses a combination of these two attention strategies, ensuring the computational complexity is linear with the input sequence length.



(a) Window attention, $w=3$ (b) Global attention, $g=1$ (c) LED

Fig. 12. Building blocks of LED attention mechanism

In this work, due to limited computing and memory resources and limited training dataset, we use pre-trained LED models on arxiv papers in scientific_papers dataset

which is for generating abstract for scientific papers. In this work, LED has been fine tuned on a downstream task to generate slides from the technical paper automatically.

The model has the vocab_size of 50,265. It uses LED Tokenizer for tokenizing the input sequences into embeddings. It uses byte-level Byte-Pair-Encoding for its tokenizer. LED Tokenizer treats space as part of the token and a word with or without space is treated differently. For summarization tasks, an LED model with a language head is used, called LEDForConditionalGeneration on huggingface. The input sequence tokens fed to the model can either be fixed length, or padded, or truncated. Due to computing resource constraints, we set the encoder maximum sequence length to 9500 tokens and decoder's maximum sequence length to 950 tokens. It uses <s> to depict the beginning of a sequence as well as separator token and </s> to depict the end of sequence. It uses <pad> for padding the sequence. The attention_mask is an array of 0s and 1s. 0s represent token indices which should be paid attention to and 1s that should be ignored. It's fed to the model to avoid attending to padded token indices and finally global_attention_mask is set depending on the downstream task. For summarization, the first token in every input sequence is set to 1. This token attends to all other tokens and all others to this one.

The LED model processes scientific documents as input sequences and generates slides as output sequences. This involves dividing each sequence into 12 attention windows, each capable of accommodating 1024 tokens. Both the encoder and decoder operate simultaneously to handle the input and output sequences.

With 12 layers in both the encoder and decoder of our model, the input and output sequences undergo 12 iterations of processing. Each encoder and decoder employs 16

attention heads, which calculate attention scores to capture diverse relationships and dependencies between the input and output sequences. These scores are then used in the Feed Forward network, which, irrespective of token representations, learns non-linear and local transformations.

The intermediate representations, each 4096 tokens in size, are produced by the self-attention mechanism and the feed-forward network. The encoder's intermediate representations aid the decoder in generating the target sequence, while the decoder's intermediate representations facilitate the generation of subsequent tokens in the target sequence.

Table 1
LEDForConditionalGeneration Architecture

Name of Parameter	Value
Model	led-large-16384-arxiv
Number of encoders layers	12
Number of decoders layers	12
Number of Attention Heads in encoder	16
Number of Attention Heads in decoder	16
Size of embeddings produced / Hidden layer size	4096
Attention Window length and its size	12 and 1024
Maximum Encoder length	16384
Maximum Decoder length	1024
Library used for implementing LED	Huggingface Transformers
Tokenizer	LED Tokenizer

Fig. 11. shows the output when the LED tokenizer is applied on one sample slide. The first token is 1 which is the token ID for bos token $\langle s \rangle$, and ends with 2 which is the token ID for eos token $\langle /s \rangle$ in LED. The attention mask determines which tokens should be attended.

Fig. 13. LED tokenizer applied to one sample paper

Name of Parameter	Value
Batch Size	1
Training data size	70%
Validation data size	15%
Test data size	15%
Optimizer	Adam
Loss function	Cross Entropy
Epochs	5

Maximum Input tokens	9500
Maximum Output tokens	950
Evaluation Metrics	Rouge Scores (R1, R2, RL)

3.6 BIGBIRD-Pegasus

BIGBIRD-Pegasus is a sequence-to-sequence language model developed by researchers at Google that has achieved state-of-the-art results on various tasks involving very long sequences such as long document summarization and question-answering with long contexts.

BigBird is a sparse-attention mechanism that aims to improve performance on various tasks that require long context [44] from quadratic complexity to linear.

Similar to LED, it also uses global and window attention patterns, but it also uses random attention, in addition to that. It comes in two implementations, `original_full` for sequence length less than 1024 and `block_sparse` for longer sequences. In this study, `block_sparse` attention mechanism due to large input sequence length.

It's a combination of three types of attention, i.e., global, window and random attention. It uses a global attention pattern to support long-range dependencies. All tokens attend to global tokens and vice versa. It uses a local or sliding window to support non-local interactions; few tokens are randomly selected. It's based on random graphs.

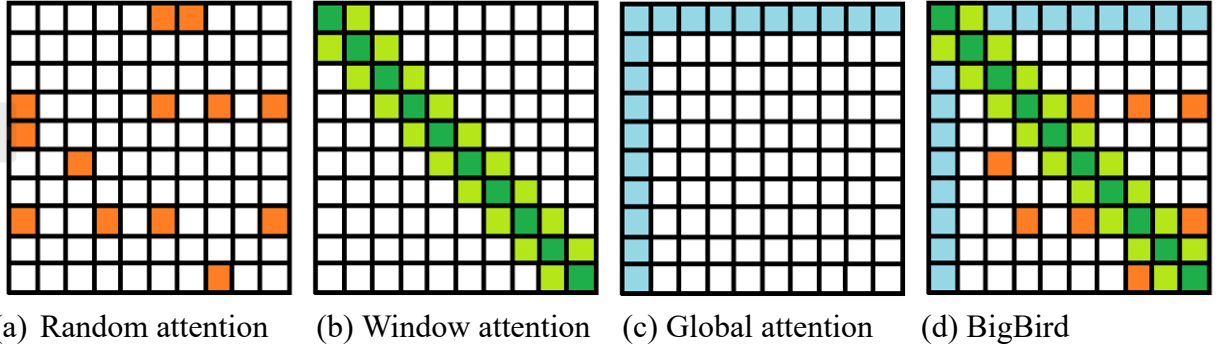


Fig. 14. Building blocks of BigBird attention mechanism

BigBird uses a state-of-the-art pretrained abstractive summarizer called Pegasus for Summarization.

Pegasus is specifically pre-trained to achieve various objectives for abstractive summarization. It has been evaluated on 12 downstream summarization tasks like news, science, stories, but most importantly on scientific articles arxiv dataset to generate abstract from the paper body.

BIGBIRD-Pegasus uses Pegasus Tokenizer based on the SentencePiece tokenizer. It treats the input as a raw input stream and doesn't use space for separating words, unlike other tokenizers.

We use pre-trained BIGBIRD-Pegasus on scientific paper dataset sourced from arXiv to save compute and due to limited training data. It has a vocab size of 96,103. It has an end of sequence token `</s>`. It uses `<pad>` for padding the input. For summarization tasks, a BIGBIRD-Pegasus model with a language head is used, called `BigBirdPegasusForConditionalGeneration` on huggingface. The input sequence tokens fed to the model can either be fixed length, or padded, or truncated. Due to computing resource constraints, we set the encoder maximum sequence length to 4096 tokens and

decoder's maximum sequence length to 1024 tokens. The attention_mask is an array of 0s and 1s. 0s represent token indices which should be paid attention to and 1s that should be ignored. It's fed to the model to avoid attending to padded token indices.

Table 3
BigBirdPegasusForConditionalGeneration architecture

Name of Parameter	Value
Model name	bigbird-pegasus-large-arxiv
Number of encoders layers	16
Number of decoders layers	16
Number of Attention Heads in encoder	16
Number of Attention Heads in decoder	16
Size of embeddings produced / Hidden layer size	4096
Attention type	Block sparse
Block size	64
Maximum Encoder length	4096
Maximum Decoder length	4096
Library used for fine-tuning LED	Huggingface Transformers
Tokenizer	PegasusTokenizer

Fig. 13. shows the output when the Pegasus tokenizer is applied on one slide sample record. The sequence ends with 1, which is the token ID for </s> which is an eos token in BIGBIRD-Pegasus. It uses 0 as a padding token ID.

[illegible]

Fig. 15. Big Bird Pegasus tokenizer applied to one sample paper

Table 4
Our Fine-tuned model architecture

Name of Parameter	Value
Batch Size	1
Training data size	70%
Validation data size	15%
Test data size	15%
Optimizer	Adam
Loss function	Cross Entropy
Epochs	3
Metrics	Rouge Scores F1 scores(R1, R2, RL)

3.7 Evaluation Metrics

One of the automated evaluation metrics is the ROUGE(Recall-Oriented Understudy for Gisting Evaluation) score, which assesses how much the n-grams in a generated summary overlap with those in one or more reference summaries created by humans [45]. For instance, ROUGE-2 looks at the overlap of 2-grams between a summary generated by a system and a reference summary [45]. Common versions used in prior studies include ROUGE-1, ROUGE-2, and ROUGE-L, with ROUGE-L measuring the longest common subsequence, which is a continuous sequence of words shared between the generated summary and the reference. Given its widespread use in assessing abstractive summarization models in research papers, it's a suitable metric for comparing different models [13], [14].

We use evaluate function from huggingface library to compute the rouge score for comparing the predicted and golden truth text, which returns R1, R2 and RL.

3.8 Experimental Platform

The experiments were conducted using a single NVIDIA GPU (Tesla P100) with 40960MiB memory on the San Jose State College of Engineering High Performance Computing (HPC) cluster.

4. EXPERIMENTAL RESULTS

4.1 Text pre-processing

Text processing holds significant importance in the context of model training, prompting us to conduct a range of experiments prior to the fine-tuning stage.

Our research found that common text normalization methods, like removing stopwords, lemmatization, and stemming, did not work well with our scientific and technical dataset (see Table 5.). Stemming and lemmatization in particular, lead to lower performance because our dataset contains specialized vocabulary and domain-specific terms not found in standard dictionaries. These terms are crucial for conveying the precise details in scientific texts. In our experiments with a dataset that included stopwords, we further confirmed this by observing a significant improvement in ROUGE scores when we avoided normalization techniques. We utilize a pre-trained large LED model on the arXiv papers in `scientific_papers` dataset as it's capable of handling long documents up to 16384 tokens. However, we choose to limit it to 9,500 tokens to align with our available computing resources.

Table 5
Results after Text normalization techniques on LED

Technique	R1	R2	RL
Stopwords Removed & Lemmatization	32.52	7.25	12.2
Stopwords Removed & Stemmed	31.6	6.6	11.8
Without Normalization	36.08	8.9	12.39

4.2 Slide Stemming

Additionally, we performed pre-processing on the slide decks in our training and test sets. We conducted Slide Stemming Slide Stemming, which is explained in Section 3.3.8, to find nearly identical slides in the dataset at duplication levels of 70%, 80%, and 90% (as mentioned in Section 3.3.8, animations, etc., often lead to highly similar slides that we call duplicates). We've presented the outcomes in Table 6. We noticed significant increases in ROUGE scores, with improvements in performance at each threshold level. At a cosine similarity threshold of 90%, the R1 is 36.08, R2 is 8.9 and RL is 12.39.

Table 6
Results after Slide Stemming on LED

Content Similarity Threshold	R1	R2	RL
70%	32.52	7.25	12.2
80%	31.6	6.6	11.8
90%	36.08	8.9	12.39

Based on these results in Table 6, in our slide generation experiments, we use a 90% content similarity threshold.

4.3 Matching Algorithm Experiments on pre-trained LED model

We propose several different techniques as described in Section 2.5 and Section 3.3.9, for enhancing the basic matching algorithm used to match document text to its corresponding slides.

To handle limited computing resources for lengthy scientific tokens documents and model sequence length constraints, we segment documents and match sections with slides, treating slides as abstract paper summaries.

We match paper sections with the most suitable slides based on the textual embeddings by generating them with a pre-trained Sentence Transformers model.

We experiment with three approaches to match paper section to a slide:

4.3.1 Matching with highest cosine similarity score

As shown in Table 7, when we match sections with slides based solely on the highest cosine similarity score, we achieve a ROUGE score of 24.3 for R1, 10.2 for R2, and 18.7 for RL.

Table 7
Results from Section-Slide matching on Maximum Cosine Score

Cosine Similarity threshold	R1	R2	RL
Max. score	24.3	10.2	18.7

4.3.2 Matching based on cosine threshold

To improve match quality, we introduce a threshold value in our approach, determining acceptable matches. We noticed that we increased the threshold: higher values led to the removal of more papers, e.g. 0.4 removed 13%, 0.45 removed 17.5%, and the trend continued, e.g. 0.5 removed 25%. As we increased the threshold value, we noticed the Rouge scores showed no significant improvements, so we made the decision to proceed with a threshold value of 0.4 for training the model, as it removed the least amount of papers.

Table 8 reveals that when we set the cosine similarity threshold value to 0.4, we obtain a ROUGE score of 24.3 for R1, 10.2 for R2, and 18.7 for RL. When we increase the threshold to 0.6, the ROUGE scores only marginally improve to 25.75 for R1, 9.27 for R2, and 19.14 for RL. Since this increase is not substantial, we opt to retain the cosine similarity threshold for further experiments at 0.4 to preserve the data length.

Table 8
Results from Section-Slide matching on Cosine threshold value

Cosine Similarity threshold	R1	R2	RL
0.4	24.5	10.03	18.9
0.45	24.54	9.81	18.75
0.5	24.2	9.32	18.4
0.55	25.5	9.64	19.14
0.6	25.75	9.27	19.14

4.2.3 Matching in a hierarchical manner

Our third approach delves into the alignment of the slide deck structure with the format of scientific papers, shedding light on the organization of slide decks. While we observed only marginal changes in ROUGE scores, this investigation led us to the conclusion that, for our specific dataset, the content organization of slide decks aligns with that of the corresponding scientific documents. As Table 9 reveals that the Rouge scores don't see any significant improvements compared to the Rouge scores in Table 8 for cosine threshold value of 0.4, where ROUGE score improved by 3.4% , 5.7% and 3.7% for R1, R2, and RL respectively.

Table 9
Results from Section-Slide matching hierarchically

R1	R2	RL
25.35	10.6	19.6

4.4 Fine-tuning the Pre-trained models

We chose to start from publicly available checkpoints of the LED and BIGBIRD-Pegasus models on Hugging Face, which were originally trained on the arXiv dataset of scientific papers. We fine-tuned both the models on our downstream task, as training a model from ground up wasn't feasible due to limited training data and computing resources.

The arxiv papers from scientific_papers dataset available on Hugging Face, is a valuable resource for researchers and developers working in the field of natural language processing (NLP). It provides a diverse collection of scientific papers from the arxiv preprint server, which covers a wide range of academic disciplines, including physics, computer science, mathematics, and many others.

The dataset contains 203,037 training instances, 6,436 validation, and 6,440 test data. It has three features: article, abstract and section names. This dataset can be used to generate an abstract from long documents like scientific documents and predict the section names.

To tackle the challenge of insufficient data for training a transformer-based model for slide generation, we turn to pre-trained models on this dataset. This is because our chosen

dataset for slide generation lacks the volume needed to train a transformer model effectively.

The dataset encompasses papers from various academic fields, including computer science. This is particularly relevant for PS5K, which includes papers from computational linguistics, systems, and system security. Leveraging the diversity of this dataset, we fine-tune pre-trained models on the PS5K dataset to facilitate automatic slide generation.

Our experimental design can be categorized into two main groups: processing complete long documents and processing segmented documents. We describe both in the rest of this section.

4.4.1 Abstractive approach of slide generation

In this category, we employ advanced pre-trained large sequence-to-sequence models like LED and BIGBIRD-Pegasus. Our approach involves providing the complete scientific document along with the corresponding slide deck to enable the model to understand the context comprehensively.

As shown in Fig. 15 and 16, our pre-processed dataset typically contains scientific papers with a median token length of 6712 tokens and slides with 746 tokens. We utilize a large LED model, which can handle up to 16,384 tokens. However, due to our limited computing resources, we train our model on 9500 tokens for scientific documents and 950 tokens for its slides. Both of these values surpass the median token length of our processed dataset.

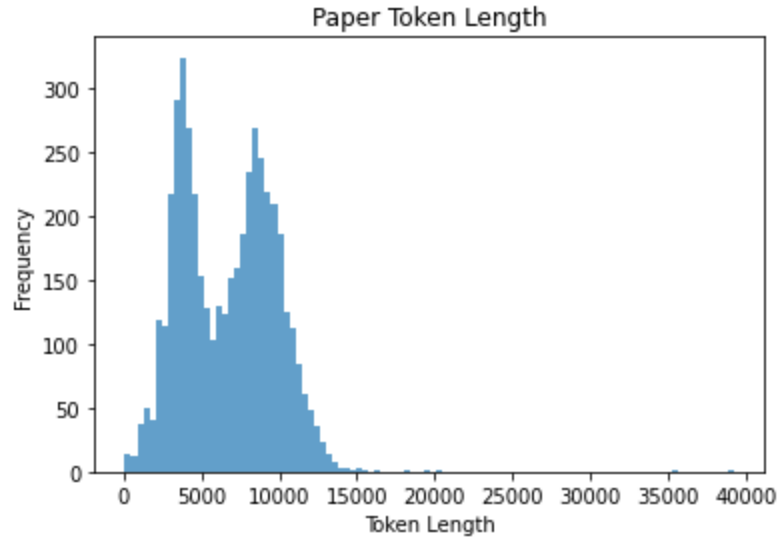


Fig. 16. Distribution of pre-processed scientific document token length in PS5K

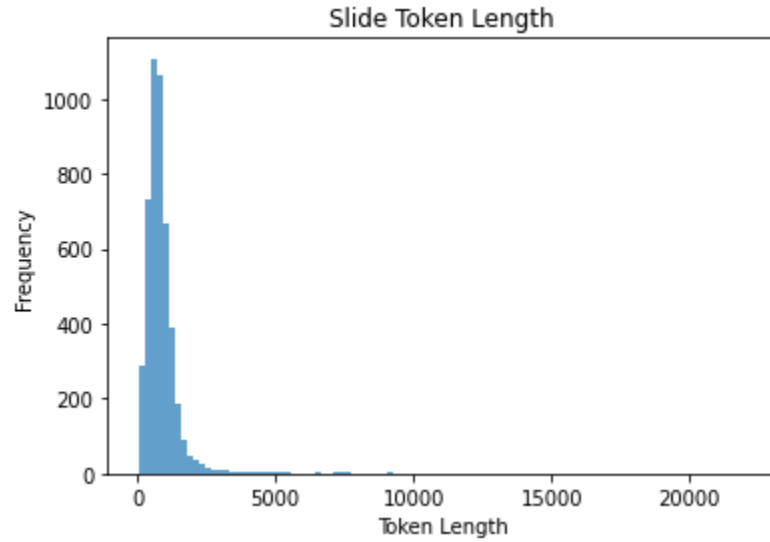


Fig. 17. Distribution of pre-processed slide token length in PS5K

In contrast, BIGBIRD-Pegasus can only handle 4096 tokens for both the encoder and decoder. The 4096 tokens for the scientific documents are shorter than the median token length of our dataset's papers. Because both LED and BIGBIRD-Pegasus use self-attention in the decoder, a larger attention size at the decoder demands substantial computational resources, as the time and memory complexity increases quadratically

with respect to the input sequence length. Therefore, we decided to train BIGBIRD-Pegasus with 4096 tokens for the encoder and 1024 tokens for the decoder.

Unlike LED, we fine-tune BIGBIRD-Pegasus by filtering scientific documents with a token length of less than 4000 tokens and slides with a token length of less than 1024 tokens. We made this decision because BIGBIRD-Pegasus has a maximum token length that is shorter than the median token length of the dataset, as shown Fig. 16 and Fig. 17. Instead of truncating most of the training instances, we opted to train the model based on its token length capacity.

As demonstrated in Table 10, LED outperforms BIGBIRD-Pegasus on PS5K. This can be attributed to LED's capability to handle longer input sequences and higher volume training dataset compared to BIGBIRD-Pegasus, which, due to its shorter input sequence capacity and shorter training dataset, had to train on a 6x smaller dataset. This could have also impacted the model's performance. Despite experiencing significant drops in R1 and R2 scores, with reductions of 45.16% and 47.03%, BIGBIRD-Pegasus still managed to achieve a respectable RL score compared to LED, which only saw a decrease of 20.07%. This suggests that BIGBIRD-Pegasus maintains the order and sequence of words in the generated text effectively.

Table 10
Results after model evaluation on complete scientific document

Model Name	R1	R2	RL
LED	37.44	8.93	13.80
BIGBIRD-Pegasus	20.53	4.73	11.03

4.4.2 Segmentation of Scientific Documents into Sections

While large pretrained transformers have proven highly effective in generative tasks, addressing long sequence inputs remains a notable challenge, particularly in tasks such as lengthy text summarization. Training these models is resource-intensive, and scientific documents, often surpassing the maximum input context of most models, present an additional complexity. In response, we opted to segment these documents into their constituent sections, considering each section as an abstractive summary matched to a slide based on textual similarity scores.

We explored three distinct segmentation strategies to align paper sections with slides prior to model training, as described in Section 2.6, 3.3.9 and 4.2. Our findings suggest that the performance of models when provided with whole-document inputs is generally better compared to document segments. This variation in performance can be attributed to factors such as limited training data and some misalignment between scientific documents and slides.

When the model processes an entire document, it benefits from a more comprehensive contextual understanding. In contrast, when dealing with document segments, it focuses on only matched section-slide pairs and requires no truncation in the input sequence parts of the document, which not only increase training data volume but also help the model focus on all the important parts of input document, which is crucial for generating slides as a section can potentially have one or more slides.

Table 11. demonstrates LED's performance, achieving a higher R2 and RL ROUGE scores by 47% and 45% respectively, as compared to LED model's results on processing the complete scientific document and slides, as shown in Table 12.

Table 11
Results after model evaluation on slide-section matching on LED

Model Name	R1	R2	RL
LED	30.9	17.0	25.5

Table 12
Results after model evaluation on complete scientific document(repeated)

Model Name	R1	R2	RL
LED	37.44	8.93	13.80
BIGBIRD-Pegasus	20.53	4.73	11.03

4.4.3 Comparison with GPT model i.e. gpt-3.5-turbo

To benchmark our results against the latest GPT model, gpt-3.5-turbo, which supports a maximum token length of 4096 tokens. We filtered out papers with fewer than 4096 tokens and then proceeded to validate 153 pairs of papers and slides from a validation set of 702 paper-slide pairs. As shown in Table 13, we observed decreases in multiple ROUGE metrics when compared to Table 11. Specifically, R1 decreased by 22%, R2 by 18.5% and R3 by 10.9% for the LED model. Notably, when compared with BIGBIRD-Pegasus, R1 and R2 exhibited better Rouge scores with increases of 42.23% and 35.2%, while RL only dropped by 11%.

Table 13
Results after model evaluation on complete scientific document on gpt-3.5-turbo

Model Name	R1	R2	RL
gpt-3.5-turbo	29.2	7.3	12.29

4.5 Comparison to Previous work

Comparing our work to prior studies [9](refer Fig. 18.), it becomes evident that extractive models consistently outperform abstractive models. This observation holds true even when considering the performance of models like GPT. However, it is crucial to consider that previous document-level experiments are not directly comparable to our study. The first three involved unsupervised approaches, while the latter three employed extractive summarizers that were trained from the ground up for 10 epochs. In contrast, we employed a supervised learning approach with pre-trained abstractive models due to constraints of our high performance cluster usage. Our decision to conduct experiments on abstractive summarizers stemmed from the substantial body of research that has already explored extractive summarizers. Additionally, prior work, including the same study mentioned in Fig. 18. below, has reported that while abstractive approaches may not achieve Rouge scores as high as those of extractive methods, they tend to generate less verbose text.

In the context of section-level analysis, despite the lack of detailed information on text preprocessing in previous studies, we observed a significant improvement in our R2 scores compared to any of the previous experiments. Furthermore, our RL scores were on par with the section-level results from previous research. This leads us to the conclusion

that our hybrid approach, involving training the model on section-slides with a cosine threshold value for matching sections to slides, played a crucial role in enhancing R2 scores. This is in contrast to prior work where they simply matched the slide section based on the highest cosine similarity. We believe our approach is effective because even the highest cosine similarity score may be relatively low for matching, as the content of slides is not solely derived from the paper, and not every section corresponds to a slide match. However, our R1 scores did not reach the same level as those achieved in previous section-level experiments.

Furthermore, our work includes Rouge F1 scores, which provide a harmonic mean of precision and recall of Rouge scores. This differs from previous studies, where only recall Rouge scores were presented, with no explanation for the focus solely on recall scores.

	Models	R-1	R-2	R-L	T(h)
Document Level	Lead20%	37.68	6.62	15.90	-
	Sefid et al. [9]	36.33	8.73	17.02	-
	TextRank [5]	38.87	9.28	19.75	-
	SummaRuNNer [6]	45.04	11.67	23.03	18
	SummaRuNNer+Attn [1]	47.49	11.67	22.89	38
	BertSum	52.34	15.06	36.87	42
Section Level	BART	46.34	11.14	29.85	20
	BART+Empty summary	43.17	10.25	27.47	20
	T5	44.72	10.23	29.63	18

Fig. 18. Results from previous work[9]

5. DISCUSSIONS

We fine-tuned the abstractive summarizers LED and BIGBIRD-Pegasus and evaluated them in the context of two approaches to dataset pre-processing. In the first approach, we employed the entire scientific document to instruct the model in generating slides. Table 14 mentions that LED outperformed the BIGBIRD-Pegasus model, achieving superior R1 and R2 scores while maintaining comparable RL scores. This suggests that, although BIGBIRD did not attain the same R1 and R2 scores as LED, it demonstrated a similar level of coherence, as indicated by the RL scores. It's crucial to recognize that models with an extended attention range exhibited enhanced performance. However, it is important to note that BIGBIRD-Pegasus is a heavier model with 577 million parameters, compared to LED's 460 million parameters. Furthermore, BIGBIRD was trained on a reduced dataset due to its shorter maximum sequence length support of 4096 tokens, as opposed to LED, which was trained on 9500 tokens with a maximum sequence length of 16k. This indicates a demand for more extensive training data and additional training time.

In our second approach, we exclusively supplied the models with scientific document sections possessing a cosine similarity score greater than or equal to 0.4. This experiment yielded superior results compared to the first approach, underscoring the advantage of inputting filtered sections. These sections are shorter and do not necessitate truncation. Consequently, the model concentrates solely on abstractive summarization, diverging from the first approach, where it learned to select significant sections and generate slide-like text.

Table 14
Results from Fine-tuned models

Type	Model Name	R1	R2	RL
Abstractive	LED	37.44	8.93	13.80
	BIGBIRD-Pegasus	20.53	4.73	11.03
	gpt-3.5-turbo	29.2	7.3	12.29
Hybrid	LED	30.9	17.0	25.5

6. LIMITATIONS AND FUTURE WORK

In this section, we address open issues and outline potential areas for future research. Our study possesses certain limitations, many of which stem from the choices we made to accommodate the available computing resources and dataset.

Our primary model, the Longformer Encoder-Decoder (LED), can handle input tokens up to 16,384 and output tokens up to 1,024, achieving state-of-the-art results in text summarization. However, we decided to restrict input tokens to 9,500 and output tokens to 950 to match our computing resources.

Similarly, BIGBIRD-Pegasus can accommodate up to 4,096 tokens for both input and output. Yet, we chose to limit the output token length to 1,024, aligning with our available computing resources. This decision resulted in truncation of both the scientific document and its corresponding slide text. This approach may not be ideal for our task, as after text preprocessing, the median token length for scientific documents is 6,712 tokens, extending to 90th percentile token length of 10,418. For slides, the median token length is 746 tokens, extending to 1,379 tokens at the 90th percentile. As a result, it's possible that the model hasn't comprehensively processed the entire input and output sequences. Generating slides, unlike typical text summarization, requires the model to grasp the complete scientific document. This is because scientific documents exhibit a distinctive hierarchical structure, and each section within the paper can potentially serve as a source for generating a slide. Consequently, the model must learn which sections hold significance and how to create abstract summaries suitable for slide-like content.

Also, due to limited computing resources, we couldn't perform an extensive hyper-parameter tuning, as unlike model training, the hyper-parameter tuning needs more memory. Therefore, we had to manually try with different hyperparameters values to adjust the results.

Due to the limited availability of datasets for scientific document tasks, particularly for generating slides, we had to work with a dataset of just 5,000 paper-slide pairs in PS5K. This decision to use this dataset was also influenced by the fact that the text extraction of the slide and papers into XML files was done and didn't require additional work, unlike another available dataset[32], where all the slides were image files and required sophisticated tools to extract the slide content.

Model training demands substantial data and computational resources, which prompted us to fine-tune pre-trained models originally trained on arxivs papers from scientific_papers dataset on huggingface. This dataset comprises 203,037 samples of articles, abstracts, and section names. However, these models were initially designed to generate abstracts for articles, a task that differs from creating slide content. Slide content tends to follow a hierarchical structure and can be longer than typical abstracts, which were limited to 512 tokens. Our decision to fine-tune these models was based on two primary reasons. First, we aimed to enhance the models' familiarity with the vocabulary commonly found in scientific documents, especially those in the field of computer science. Second, we treated the task of slide generation as an abstractive summary of various sections within a scientific document. While this approach helped us save both time and memory, it remains uncertain whether the models' prior training on scientific

papers positively influenced their fine-tuning for this specific task. This work can be enhanced by training the models from ground up on LED and BIGBIRD-Pegasus.

We noticed that our abstractive summarizers for processing the entire long scientific document create verbose slides with full sentences. This work can be enhanced by paraphrasing the output into phrases.

Due to the absence of funding for this thesis, our evaluation of the model's performance was limited to automatic assessments based on ROUGE scores. While ROUGE scores serve as a fundamental evaluation metric for many transformer-based models like LED and BIGBIRD-Pegasus, relying solely on automated evaluation is inadequate for tasks in Natural Language Processing, such as summarization. To ensure a more comprehensive evaluation of the model's performance, this work could benefit from human feedback. Reviewers with a strong understanding of the field can assess the content in the scientific document and identify key concepts suitable for inclusion in the slides.

7. CONTRIBUTIONS

In this section, we outline the key contributions of our study in the context of automatically generating slides from lengthy documents, such as scientific papers. Our primary focus is on processing long documents, particularly scientific papers, to create presentation slides for knowledge workers. Given the subjectivity involved in organizing slide content, we maintain human involvement in the process, treating these slides as preliminary drafts.

To achieve this, we consider the slides as abstractive summaries of the sections within scientific documents. We fine-tuned large language models, including the Longformer Encoder-Decoder (LED) and BIGBIRD-Pegasus, renowned for their state-of-the-art performance in text summarization. These models were initially pre-trained on arXiv papers from scientific datasets available on Hugging Face and were further fine-tuned on the PS5K dataset, one of the largest datasets available for the task of automatically generating slides from scientific papers.

Before fine-tuning the models, we conducted a series of experiments involving various text preprocessing techniques. Notably, we observed that our selected models performed optimally without the need for any normalization techniques. Additionally, we introduced Slide Stemming to remove near-duplicate slides and those with minimal content, aligning with the desired slide format of having both a title and text. This step yielded significant improvements in our results. We explored the impact of varying content similarity thresholds between consecutive slides, testing values of 70%, 80%, and 90%.

Given the pivotal role of preprocessing in model performance and the absence of comprehensive insights on these techniques in existing literature, our research delves into the effects of these methods on the PS5K dataset. In Section 4.1 and 4.2, we present the results of our extensive experiments.

As the field of automatically generating slides from lengthy documents gains traction within Natural Language Processing, we explore a variety of techniques to achieve our objectives while adhering to the constraints posed by our available computing resources and dataset volume.

We approach the task of processing lengthy documents, particularly scientific papers, from two distinct angles. Firstly, we process the complete scientific document in an effort to train the model to generate slide-like content effectively. In this pursuit, we fine-tuned both LED and BIGBIRD-Pegasus, and our findings indicate that LED generated better Rouge scores with R1, R2 and RL higher by 45.16%, 47.03% and 20.07% respectively from BIGBIRD-Pegasus Rouge scores.

Secondly, in light of computing constraints and the limitations of large language models in handling very lengthy scientific documents, we adopted a segmentation approach for these documents. The segmentation process ensures that all segments are arranged in a sequential order, with due consideration to maintaining boundaries to avoid splitting text in the middle. Our segmentation divides the scientific documents into their respective sections and aligns these sections with the corresponding slides. We consider each slide as an abstractive summary of a section within the scientific document.

In aligning the sections with the slides, our work explores various strategies that rely on textual similarity. Initially, we align sections with slides based on the highest cosine similarity score. Subsequently, we introduced a cosine threshold value as a precondition for aligning sections and slides. We conducted experiments with different threshold values, ranging from 0.4 to 0.6 in increments of 0.05. We observed that increasing the threshold value led to the exclusion of section-slide pairs from the dataset without a significant improvement in evaluation metrics, specifically ROUGE scores.

Lastly, we aimed to gain insights into the organization of slides within the PS5K dataset. While maintaining the cosine threshold value to retain dataset volume, we reorganized section-slide pairs in sequential order, ensuring that subsequent slides had the same or greater order than the previous slide. We extracted the top three slide matches for each section and determined the index for better alignment in a sequential manner to align the sections and slides. Our experiments shed light on how slides are structured in this dataset, and we observed a slight improvement in our evaluation metrics for all three types of ROUGE scores, demonstrating that in our dataset, slides are organized around the scientific document's sections.

As shown in the preceding explanation, our study brings to light various effects of text preprocessing techniques on the PS5K dataset. It addresses the task of automatic slide generation from long documents, such as scientific papers, in two distinct approaches using state-of-the-art large language models like LED and BIGBIRD-Pegasus, and share the results.

8. CONCLUSIONS

The increasing number of scientific documents published each year makes it challenging for knowledge workers to keep up with current and new research. Presentation slides are often the preferred method for disseminating work in conferences and seminars, as they can quickly summarize the content. However, there is a lack of tools to automatically generate presentation slides from scientific documents, and this has garnered interest in the research community. This task relies on the availability of datasets and large language models capable of processing lengthy documents and mastering the art of creating slides.

To address this challenge, we utilized the PS5K dataset to fine-tune pre-trained models for slide generation. We employed two approaches. First, we fed the complete scientific document and slides to the models to teach them how to create slides. Second, to work within the computational constraints and token limits of large language models for scientific documents, we segmented the scientific document based on its sections. We utilized two state-of-the-art large language models, LED and BIGBIRD-Pegasus. With LED, we observed that when fed the entire document as opposed to paper sections, it achieved a higher R1 score, indicating that the generated slide text had more matching unigrams with reference slides. However, R2 and RL scores were higher for section-based processing, suggesting that section-based processing maintained the sequence of bigrams and the longest common sequence, resulting in more coherent slides compared to processing the entire document. LED's cross-attention allowed it to keep track of different scientific documents even when fed sections separately. Our

experiments on LED model for inputting the sections and slide pairs performed better compared to inputting the complete document as the R2 cores were higher by 47% and RL scores were higher by 45%, with a drop in R1 scores by 17%.

As BIGBIRD-Pegasus has a smaller token limit compared to LED and scientific documents often exceed this limit, we trained BIGBIRD-Pegasus only on papers with fewer than 4000 tokens. Due to the self-attention mechanism in the decoder, we limited it to 1024 tokens to manage computing constraints. Unlike LED, BIGBIRD-Pegasus did not learn to create overview slides, but it still managed to create coherent slides, as reflected in its respectable RL score compared to LED, despite being trained on a smaller dataset. Regardless of the model or training approach used, both models lacked details and focused on creating general, abstractive summary slides. Our study shares results and follows a multifaceted approach to automatically generate presentation slides from scientific documents using large language models like LED and BIGBIRD-Pegasus, this aspect remained uncharted in prior research endeavors.

Our findings indicate that LED generated better Rouge scores with R1, R2 and RL higher by 45.16%, 47.03% and 20.07% respectively from BIGBIRD-Pegasus Rouge scores when processing the complete scientific document and slides.

Finally, we also conducted a comparison of our results with the state-of-the-art model, gpt-3.5-turbo. Due to the lack of funding for this thesis, we couldn't perform an extensive set of experiments. Nevertheless, to assess performance, we conducted a single experiment to evaluate the gpt model's ability to generate slides when provided with a complete scientific document containing fewer than 4096 tokens. Despite the extensive

training of the gpt model on multiple tasks and datasets, our results from LED surpassed the gpt model in both approaches. When compared to BIGBIRD-Pegasus, although the gpt model had higher R1 and R2 scores, which measure unigram and bigram overlap, the RL scores showed no significant difference, indicating that our BigBirdmodel also generates coherent summary slides compared to the gpt model.

Considering the constraints of limited computing resources, the shorter original dataset, and the need to conduct some experiments on a reduced dataset to prevent data truncation, as well as our models being primarily based on pre-trained model configurations, we find the obtained results to be promising.

Literature Cited

- [1] Y. Hu and X. Wan, "PPSGen: Learning-based presentation slides generation for academic papers," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 4, pp. 1085–1097, Apr. 2015, doi: 10.1109/tkde.2014.2359652.
- [2] D. R. Radev, E. Hovy, and K. McKeown, "Introduction to the special issue on summarization," *Computational Linguistics*, vol. 28, no. 4, pp. 399–408, Dec. 2002, doi: 10.1162/089120102762671927.
- [3] W. Kryscinski, N. S. Keskar, B. McCann, C. Xiong, and R. Socher, "Neural text summarization: A critical evaluation," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, doi: <https://doi.org/10.18653/v1/d19-1051>.
- [4] R. Nallapati, F. Zhai, and B. Zhou, "SummaRuNNer: A recurrent neural network based sequence model for extractive summarization of documents," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, Feb. 2017, doi: 10.1609/aaai.v31i1.10958.
- [5] S. Narayan, S. B. Cohen, and M. Lapata, "Ranking sentences for extractive summarization with reinforcement learning," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018, doi: 10.18653/v1/n18-1158.
- [6] Y. Liu, "Fine-tune BERT for extractive summarization," 2019, *arXiv:1903.10318*.
- [7] S. Wang, X. Wan, and S. Du, "Phrase-based presentation slides generation for academic papers," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, Feb. 2017, doi: 10.1609/aaai.v31i1.10481.
- [8] S. Syamili and A. Abraham, "Presentation slides generation from scientific papers using support vector regression," *2017 International Conference on Inventive Communication and Computational Technologies (ICICCT)*, Mar. 2017, doi: 10.1109/icicct.2017.7975205.
- [9] A. Sefid, J. Wu, P. Mitra, and C. L. Giles, "Automatic slide generation for scientific papers," *International Conference on Knowledge Capture*, vol. 2526, pp. 11–16, Nov. 2019, [Online]. Available: <http://ceur-ws.org/Vol-2526/paper2.pdf>
- [10] R. Nallapati, B. Zhou, C. dos Santos, C. Gulcehre, and B. Xiang, "Abstractive text summarization using sequence-to-sequence RNNs and beyond," in *Proceedings of The 20th SIGNLL Conference on Computational Natural Language Learning*, 2016, doi: 10.18653/v1/k16-1028.

- [11] S. Chopra, M. Auli, and A. M. Rush, “Abstractive sentence summarization with attentive recurrent neural networks,” in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2016, doi: 10.18653/v1/n16-1012.
- [12] A. See, P. J. Liu, and C. D. Manning, “Get to the point: Summarization with pointer-generator networks,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, doi: 10.18653/v1/p17-1099.
- [13] A. Celikyilmaz, A. Bosselut, X. He, and Y. Choi, “Deep communicating agents for abstractive summarization,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, 2018, doi: 10.18653/v1/n18-1150.
- [14] J. Zhang, Y. Zhao, M. Saleh, and P. J. Liu, “Pegasus: Pre-training with extracted gap-sentences for abstractive summarization,” in *Proceedings of the 37th International Conference on Machine Learning*, pp. 11328–11339, Nov. 2019.
- [15] U. Masao and H. Kôiti, “Automatic slide presentation from semantically annotated documents,” in *Proceedings of the Workshop on Coreference and its Applications - CorefApp '99*, 1999, doi: 10.3115/1608810.1608816.
- [16] T. Shibata and S. Kurohashi, “Automatic slide generation based on discourse structure analysis,” *Lecture Notes in Computer Science*, pp. 754–766, 2005, doi: 10.1007/11562214_66.
- [17] A. Sefid, P. Mitra, and L. Giles, “SlideGen,” in *Proceedings of the 21st ACM Symposium on Document Engineering*, Aug. 2021, doi: 10.1145/3469096.3474939.
- [18] T. Hayama, H. Nanba, and S. Kunifuji, “Alignment between a technical paper and presentation sheets using a hidden markov model,” in *Proceedings of the 2005 International Conference on Active Media Technology, 2005. (AMT 2005).*, doi: 10.1109/amt.2005.1505278.
- [19] M.-Y. Kan, “SlideSeer,” in *Proceedings of the 7th ACM/IEEE-CS joint conference on Digital libraries*, Jun. 2007, doi: 10.1145/1255175.1255192.
- [20] B. Beamer and R. Girju, “Investigating automatic alignment methods for slide generation from academic papers,” in *Proceedings of the Thirteenth Conference on Computational Natural Language Learning - CoNLL '09*, 2009, doi: 10.3115/1596374.1596395.
- [21] D.-W. Li, D. Huang, T. Ma, and C.-Y. Lin, “Towards topic-aware slide generation for academic papers with unsupervised mutual learning,” in *Proceedings of the AAAI*

- Conference on Artificial Intelligence*, vol. 35, no. 15, pp. 13243–13251, May 2021, doi: 10.1609/aaai.v35i15.17564.
- [22] S. Teufel and M. Moens, “Summarizing scientific articles: Experiments with relevance and rhetorical status,” *Computational Linguistics*, vol. 28, no. 4, pp. 409–445, Dec. 2002, doi: 10.1162/089120102762671936.
- [23] A. Elkiss, S. Shen, A. Fader, G. Erkan, D. States, and D. Radev, “Blind men and elephants: What do citation summaries tell us about a research article?,” *Journal of the American Society for Information Science and Technology*, vol. 59, no. 1, pp. 51–62, Oct. 2007, doi: 10.1002/asi.20707.
- [24] A. Cohan, L. Soldaini, and N. Goharian, “Matching citation text and cited spans in biomedical literature: a search-oriented approach,” in *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2015, doi: 10.3115/v1/n15-1110.
- [25] V. Qazvinian *et al.*, “Generating extractive summaries of scientific paradigms,” *Journal of Artificial Intelligence Research*, vol. 46, pp. 165–201, Feb. 2013, doi: 10.1613/jair.3732.
- [26] A. Cohan and N. Goharian, “Scientific article summarization using citation-context and article’s discourse structure,” in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 2015, doi: 10.18653/v1/d15-1045.
- [27] C. Slamet, A. R. Atmadja, D. S. Maylawati, R. S. Lestari, W. Darmalaksana, and M. A. Ramdhani, “Automated text summarization for indonesian article using vector space model,” *IOP Conference Series: Materials Science and Engineering*, vol. 288, p. 012037, Jan. 2018, doi: 10.1088/1757-899x/288/1/012037.
- [28] J. Zhu, H. Li, T. Liu, Y. Zhou, J. Zhang, and C. Zong, “MSMO: Multimodal summarization with multimodal output,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, doi: 10.18653/v1/d18-1448.
- [29] H. Li, J. Zhu, J. Zhang, X. He, and C. Zong, “Multimodal sentence summarization via multimodal selective encoding,” in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, doi: 10.18653/v1/2020.coling-main.496.
- [30] T.-J. Fu, W. Y. Wang, D. McDuff, and Y. Song, “DOC2PPT: Automatic presentation slides generation from scientific documents,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 1, pp. 634–642, Jun. 2022, doi: 10.1609/aaai.v36i1.19943.

- [31] A. Sefid *et al.*, “Cleaning noisy and heterogeneous metadata for record linking across scholarly big datasets,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 9601–9606, Jul. 2019, doi: 10.1609/aaai.v33i01.33019601.
- [32] G. Cabanac *et al.*, “Overview of the CL-SciSumm 2016 shared task,” in *Proceedings of the Joint Workshop on Bibliometric-enhanced Information Retrieval and Natural Language Processing for Digital Libraries (BIRNDL)*, pp. 93–102, Jun. 2016, [Online]. Available: <https://aclanthology.org/W16-1511>
- [33] M. Yasunaga *et al.*, “ScisummNet: A large annotated corpus and content-impact models for scientific paper summarization with citation networks,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, pp. 7386–7393, 2019.
- [34] G. Lev, M. Shmueli-Scheuer, J. Herzig, A. Jerbi, and D. Konopnicki, “TalkSumm: A dataset and scalable annotation method for scientific paper summarization based on conference talks,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, doi: 10.18653/v1/p19-1204.
- [35] N. Meuschke, A. Jagdale, T. Spinde, J. Mitrović, and B. Gipp, “A benchmark of PDF information extraction tools using a multi-task and multi-domain evaluation framework for academic documents,” *Lecture Notes in Computer Science*, pp. 383–405, 2023, doi: 10.1007/978-3-031-28032-0_31.
- [36] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, doi: 10.18653/v1/d19-1410.
- [37] A. Vaswani *et al.*, “Attention is all you need” in *31st Conference on Neural Information Processing Systems (NIPS 2017)*, in *Advances in Neural Information Processing Systems*, vol. 30, 2007.
- [38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, vol. 1, pp. 4171–4186, Jun. 2019, doi: 10.18653/v1/N19-1423.
- [39] M. Lewis *et al.*, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, doi: 10.18653/v1/2020.acl-main.703.

- [40] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, “Pre-trained models for natural language processing: A survey,” *Science China Technological Sciences*, vol. 63, no. 10, pp. 1872–1897, Sep. 2020, doi: 10.1007/s11431-020-1647-3.
- [41] P. Manakul and M. Gales, “Long-span summarization via local attention and content selection,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, doi: 10.18653/v1/2021.acl-long.470.
- [42] S. Rothe, S. Narayan, and A. Severyn, “Leveraging pre-trained checkpoints for sequence generation tasks,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 264–280, Dec. 2020, doi: 10.1162/tacl_a_00313.
- [43] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020, *arXiv:2004.05150*.
- [44] M. Zaheer *et al.*, “Big bird: transformers for longer sequences,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, Art. no. 1450, Dec. 2020.
- [45] C.-Y. Lin and F. J. Och, “Automatic evaluation of machine translation quality using longest common subsequence and skip-bigram statistics,” in *Proceedings of the 42nd Annual Meeting on Association for Computational Linguistics - ACL '04*, 2004, doi: 10.3115/1218955.1219032.